UNIT 2, BUILDING INTELLIGENT AGENT WORKFLOWS

LECTURE 1

Workflow versus Agency

The most valuable decision in this whole course, and the one the internet gets wrong the most. When something should be an agent, and when a plain workflow wins.

CSE476 | Rohit Bharti

SECTION 00

A promise I made in week one

In Lecture 1, I said an agent is often the wrong choice. Today I prove it.

RECALL

Remember this, from Lecture 1

> **Can you write down the steps in advance? If yes, write them. You do not need an agent.**

I said that in the very first lecture and then moved on. This entire unit is that one sentence, taken seriously. Because knowing when NOT to build an agent is what separates someone who has read the hype from someone who can actually ship.

THE UNCOMFORTABLE TRUTH OF THIS UNIT

Most systems marketed as AI agents are not agents, and they are better for it. Today you will build the same feature three ways and see, in cold numbers, why the simplest version is usually the one to ship.

Today

- Three shapes a problem can take: workflow, router, and agent.
- The same support ticket problem, built all three ways, live.
- The real cost of each, in model calls, latency, and reliability, side by side.
- A test you can apply to any problem to pick the right shape.
- Why the exciting answer is usually the wrong one, and how to defend the boring answer in a viva.

HOLD ON TO THIS

The running example this unit is support ticket triage. A ticket comes in, it has to reach the right team. Every company has this problem, and it is the perfect lens for the workflow versus agent question.

SECTION 01

Three shapes

Every automation problem is one of these. Learn to see which.

THREE WAYS TO RUN A KITCHEN

The layman version

WORKFLOW	**The recipe card**	Every dish, same steps, same order. Chop, then fry, then plate. A new cook follows it exactly and cannot go wrong. Also cannot improvise when an ingredient is missing.
ROUTER	**The head waiter**	Reads the table, decides one thing: kitchen, or bar, or dessert station. Makes that one call, then the order follows a fixed path from there.
AGENT	**The chef improvising**	Given "make them something good", decides the dish, the method, tastes as they go, changes course. Powerful, expensive, and overkill for reheating a samosa.

THE QUESTION THAT SORTS THEM

Who decides the next step? In a workflow, you did, in advance. In a router, the model decides once, then you take over. In an agent, the model decides all the way through. That single question is the whole framework.

Now the technical version

WORKFLOW	**Fixed pipeline**	The sequence of steps is written by you, in code, at design time. The model may fill in a field, but never chooses the path. Fully deterministic and trivially testable.
ROUTER	**One decision, then code**	The model classifies the input into one of a fixed set of branches. Everything after that branch is ordinary deterministic code. One model call, bounded cost.
AGENT	**Model-driven control flow**	The model chooses which tools to call and in what order, based on what it observes. The number of steps is unknown in advance. Flexible, and expensive.

HOLD ON TO THIS

Notice the middle one. A huge share of what industry calls AI agents are actually routers: one classification, then a fixed pipeline. Being able to see that clearly is a genuinely valuable skill.

The spectrum, and where the cost lives

control handed to the model, and cost, both increase this way

THE ENGINEERING INSTINCT TO BUILD

Start at the left. Move right only when the problem genuinely forces you to. Every step right buys flexibility and costs money, latency, and testability. Reaching for the agent first is the mark of someone who has not felt the bill yet.

SECTION 02

One problem, three builds

Support ticket triage, done all three ways.

The problem

> *A support ticket comes in. Route it to one of five queues: billing, technical, account, sales, abuse.*

That is the whole task. One input, one of five outputs. Hold that in your head, because the shape of the task is going to tell us the shape of the solution.

THIRTY SECONDS, BEFORE I BUILD ANYTHING

Look at that problem. One classification into five buckets. Does that sound like it needs a system that can reason for multiple steps and use tools? Hold your answer. We are about to build all three and find out.

Build one, the workflow

```
KEYWORD_RULES = [
  ("abuse",   ("spam", "hacked", "security")),
  ("billing", ("charged", "refund", "invoice")),
  ("account", ("login", "password", "locked out")),
  ...
]

def triage_workflow(ticket):
    for queue, keywords in KEYWORD_RULES:
        if any(k in ticket.lower() for k in keywords):
            return queue
```

NO MODEL. ZERO CALLS. ZERO COST.

It is fast, free, perfectly testable, and it will give the exact same answer every single time. It is also blind to anything the keywords do not catch. For a great deal of real routing, this is genuinely all you need, and admitting that is the hard part.

Where the workflow breaks

> *"Nothing works and I want my money back."*

Is that technical, because nothing works? Or billing, because they want money back? The keyword rules will grab whichever appears first in the list, with no understanding of intent. That is the workflow hitting its ceiling.

THIS IS THE SIGNAL TO MOVE RIGHT

When the decision needs actual understanding of language, not just keyword presence, a workflow can no longer do it well. That, and only that, is your reason to bring in a model. Not because agents are exciting. Because the problem now needs judgement.

Build two, the router

```
ROUTER_SYSTEM = "Choose exactly one queue from this
  list: billing, technical, account, sales, abuse.
  Reply with only the queue name."

reply = client.chat.completions.create(...)   # ONE call
choice = reply.strip().lower()
queue = choice if choice in QUEUES else "technical"
```

ONE MODEL CALL, THEN ORDINARY CODE

The model now understands the ambiguous ticket, because it reads intent, not keywords. But it makes exactly one decision, from a fixed menu, and then your deterministic code takes over. The cost is bounded and known: one call, every time. This is the shape most real systems should be.

The line that saves the router

```
queue = choice if choice in QUEUES else "technical"
```

THE NAIVE VERSION

Trust the model

Use whatever it said as the queue. Then one day it replies "I think probably billing?" and your code routes to a queue that does not exist.

THE DEFENDED VERSION

Validate against the menu

The model was asked for one word from a fixed list. Anything not on the list is rejected and defaulted. Same principle as validating a tool name.

HOLD ON TO THIS

Anything a model produces is untrusted input. You met this in Lecture 1 with tool names. It is exactly the same rule here with a routing choice, and it will be the same rule in Unit 6 with everything.

Build three, the agent

```
AGENT_SYSTEM = "You have tools to inspect the queues
  and their policies. Decide which queue the ticket
  belongs in. Look things up if it helps."

for step in range(max_steps):        # a LOOP, like Unit 1
    message = client.chat.completions.create(
        model=model, messages=messages, tools=TOOL_SCHEMA)
    if not message.tool_calls:
        return parse_queue(message.content)
    # ... execute tools, loop again
```

THIS IS THE ONLY ONE THAT IS ACTUALLY AN AGENT

It can list the queues, read a policy, reconsider, and decide when it is done. It is exactly your tiny_agent from Lecture 1, pointed at triage. And for this simple task, all that machinery routes the same ticket to the same queue the router did, in three calls instead of one.

SAME TICKET, SAME ANSWER

The three, side by side

WORKFLOW	0 calls	billing	Free. Instant. Blind to anything the keywords miss.
ROUTER	1 call	billing	Cheap. Understands intent. One bounded decision.
AGENT	3 calls	billing	Flexible. Slower. More expensive. Same result here.

READ THE MIDDLE COLUMN, THEN THE FIRST

Same queue, every time. But zero, one, and three model calls. For a straightforward ticket, the agent paid three times the router for an identical outcome. Multiply that by a million tickets a month and the choice of shape is a line item on a budget.

SECTION 03

When the agent is right

Because sometimes it genuinely is. Here is how to tell.

When the agent actually earns its cost

UNKNOWN STEPS	You cannot write the path in advance because it depends on what you find partway through. Investigating why a system broke. Researching an open question.
MANY POSSIBLE TOOLS	The right combination of tools is different for every input, and enumerating all the branches by hand is impractical.
GENUINE OPEN ENDEDNESS	The task is "figure this out", not "classify this". The number of moves is unbounded and the model has to decide when it is finished.

NOTICE WHAT TRIAGE IS NOT

None of those describe routing a ticket into five buckets. The steps are known, the tools are few, and the task is classification, not open ended reasoning. That is precisely why the router won. Match the shape to the problem, not to the hype.

The test you can apply to anything

Three questions, in order

1. Can I write the steps in advance? If yes, build a workflow. Stop.
2. Is it one decision, then a fixed path? If yes, build a router. Stop.
3. Do the steps genuinely depend on what I find along the way? Only now, an agent.

You go down this list in order and you stop at the first yes. Reaching question three should feel like the exception, not the default.

THIS IS A VIVA QUESTION AND A JOB INTERVIEW QUESTION

For your capstone I will ask why your system is an agent and not a router. "Because the course is called agentic AI" is not an answer. Being able to walk down these three questions and justify where you stopped, that is the answer, and it is what an interviewer is really testing.

The misconception, stated plainly

WHAT THE INTERNET SELLS

Agents are the advanced, impressive choice

So a workflow must be the beginner move, and picking one means you did not understand the material or could not build the real thing.

WHAT ENGINEERS KNOW

The simplest shape that works is the senior move

Choosing a router when a router fits shows judgement. Reaching for an agent you do not need shows you have not paid a real bill yet. Simpler is harder to argue for, and usually right.

HOLD ON TO THIS

There is real courage in shipping the boring solution when a flashier one was available and you knew it was overkill. That courage is worth cultivating, and it is rewarded in every serious engineering team.

Hold on to this

Start at the simplest shape. Move toward the agent only when the problem forces you.

Three shapes, one question	Who decides the next step: you, the model once, or the model throughout.
Most AI routing is a router	One classification, then fixed code. Not an agent, and better for it.
Every step right costs money	The agent paid three times the router for the same answer here.
Defend the boring answer	Walking the three questions is the viva answer and the interview answer.

Before next session

DO	**Build all three**	Run the notebook. Then add a sixth queue of your own and update each of the three builds to handle it. Notice which was easiest to change and which was hardest.
DO	**Find the crossover**	Write three tickets: one the workflow handles fine, one that needs at least the router, and one you think would genuinely need the agent. Defend the third.
THINK	**Audit a real product**	Pick a product that claims to use AI agents. Apply the three questions. Decide honestly whether it needs an agent, or whether it is a router with good marketing.

ON THE SECOND ONE

The needs-an-agent ticket is the hard part. Most people cannot write one for triage, because triage does not need an agent. If you decide your third ticket does not really need one either, that is not a failure. That is you learning to see clearly.

Next session

Unit 2, Lecture 2

Tool Calling, in Depth

Once you have decided a problem does need a model to act, the whole thing hinges on tools. Next time we open up tool calling properly: how the model reads a schema and chooses, what the execution loop really does, and the part nobody prepares for, what happens when a tool fails halfway through. That failure handling is where real systems live or die.

COME WITH

The notebook run, your sixth queue added, and one honest audit of a product that claims to use agents.